

Gridlock 2.0 -- Approach

```
=====
GRIDLOCK 2.0 -- SOLUTION APPROACH
Spatio-temporal traffic-demand regression
Final model "BC" -- evaluation score 91.25092
=====
```

This document explains the approach, feature engineering, validation methodology, model evolution, tools, and reproducibility for the winning submission file `submission\_BC.csv`. Throughout, "evaluation score" means the reported metric score = $100 * (1 - \text{RMSE})$; higher is better.

----- 1. PROBLEM AND METRIC -----

Task: forecast per-cell traffic `demand` for a city represented as a grid of 6-character geohash cells, at 15-minute time-of-day resolution.

- * Target : `demand`, a continuous value in (0, 1]. Strictly positive, right-skewed (skew ~3.73), median ~0.048, mean ~0.094, with no zeros, no negatives, and no missing values.
- * Metric : RMSE, reported as evaluation score = $100 * (1 - \text{RMSE})$.
- * Forecast : predict day-49 DAYTIME slots (135-825 minutes since midnight) given full day 48 (all 96 fifteen-minute slots) plus the day-49 early-morning slots (0-120 minutes).

So the split is temporal and forward-looking: the model is trained on a full prior day plus the current day's morning, and must forecast the same day's daytime window.

----- 2. APPROACH OVERVIEW -----

The solution rests on three pillars:

- (a) Leakage-safe feature engineering. All target-derived statistics are fit on a REFERENCE day only (day 48). The current training day never sees its own same-slot demand. Day-over-day lag features carry the previous day's value forward into day-49 / test rows.
- (b) A daytime validation proxy. Because the training data contains no daytime labels for the forecast day, a purpose-built proxy provides the first trustworthy daytime error estimate (see Section 4).
- (c) Model-diversity blending. Independently-built gradient-boosted models are averaged to reduce variance and exploit complementary errors.

Final model = "BC" = a 50/50 average of two independently-built models, clipped to [0, 1]:

```
submission_BC = clip( 0.5 * B + 0.5 * C , 0, 1 )
```

- * B = a feature-enhanced, seed-averaged LightGBM (the v1 feature set plus two time-of-day profile features).
- * C = a proxy-gated blend of LightGBM and XGBoost
(0.4060 * LightGBM(v1) + 0.5940 * XGBoost(geohash column dropped)).

The combined model BC achieves an evaluation score of 91.25092.

3. FEATURE ENGINEERING (DETAIL)

All features are built from the raw columns (Index, geohash, day, timestamp, demand, RoadType, NumberofLanes, LargeVehicles, Landmarks, Temperature, Weather). Three groups of features are used.

3.1 Row-local features (no target use, no leakage)

- * geohash -> (lat, lon) decode. Each 6-char geohash is decoded to the latitude and longitude of its cell centre. Continuous coordinates let the trees generalise spatially to COLD-START cells (geohashes seen in test but not in train) instead of treating geohash as an opaque category only.
- * Cyclical time encodings. time-of-day in minutes and hour-of-day are each encoded as sin/cos pairs (tod_sin, tod_cos, hour_sin, hour_cos) so the model sees midnight and 23:59 as adjacent. Raw `tod`, `hour`, `minute` are also kept.
- * Static / categorical signals. NumberofLanes (numeric), Temperature, binary flags large_veh_bin and landmarks_bin, and missing-value indicator flags temp_missing, weather_missing, roadtype_missing. Native categoricals (geohash, RoadType, Weather) are handled directly by the tree models.

3.2 Leakage-safe history aggregates (fit on the REFERENCE day only)

Computed by grouping demand on the reference day (day 48):

- * Per-geohash : mean, std, median, min, max, count, plus a smoothed mean (shrunk toward the global mean for low-count cells).
- * Per-time-of-day : mean and std (tod_mean, tod_std).
- * Per-RoadType : mean (road_mean).
- * geohash frequency (geo_freq) as a popularity / support signal.

These are merged onto every frame. A spatial back-off chain handles unseen cells: per-cell -> 5-char geohash prefix -> 4-char prefix -> global mean.

3.3 The day-over-day lag (the single most informative engineered feature)

- * geo_tod_mean : the reference-day (day-48) demand at the SAME geohash and the SAME 15-minute slot. For day-49 / test rows this is literally "yesterday's value at this place and time", which is a strong predictor. About 88.9% of test cells have a day-48 same-slot observation available.
- * Back-off chain for missing lags:
geo_tod_mean -> smoothed per-geohash mean -> per-time-of-day mean
-> global mean.

(In model B the per-cell lag is left as NaN for unseen pairs and LightGBM handles the NaN natively; in model C it is explicitly back-filled along the chain above. The two designs are intentionally different to add diversity.)

3.4 Extra profiles added in model B

- * `road_tod_mean` : RoadType x time-of-day demand profile (smoothed).
 - * `lanes_tod_mean` : NumberofLanes x time-of-day demand profile (smoothed).
- These capture how each road class / lane count behaves across the day, and directly counter the covariate shift between train and test (the daytime test window is enriched in busier road classes). They were the ONLY extra feature family beyond v1 that measurably helped on the daytime proxy.

4. VALIDATION METHODOLOGY

The central difficulty: the training data has NO daytime labels for the forecast day. Day-49 training rows are night / early-morning only, and day-48 is a full day. A naive cross-validation on the available (mostly night) rows measures the wrong thing -- it rewards night-time accuracy and MISLED several early variants that looked better in night CV but did not improve the daytime forecast.

The fix was a DAYTIME PROXY:

- * Train on day-48 NON-daytime rows.
- * Predict day-48 DAYTIME rows (which DO have labels) and score them.

This proxy mimics the real task (forecast daytime from earlier data) on a window where ground truth exists, and was the first metric that moved in step with the true daytime error. It became the arbiter for feature-family selection.

Boosting round counts were calibrated on a complementary protocol, the lag-present CV-A, which evaluates with the day-over-day lag available (matching how the lag is present for test rows). The final round counts (B: 437; C LightGBM: 217; C XGBoost: 259) and the C blend weights come from this offline calibration and are then BAKED IN as constants -- there is no search at runtime.

5. MODEL EVOLUTION AND FINAL SELECTION

Progression of candidates (evaluation score in parentheses):

v1	single LightGBM, v1 feature set	90.791	
v3	added morning / shift features	90.678	(worse)
v4	10-seed bag	90.689	(flat)
v5	seed-averaged variant	90.689	(flat)
C	LightGBM + XGBoost proxy-gated blend	91.186	
BC	50/50 average of B and C	91.25092	(SELECTED)

What HELPED:

- * Model diversity. Combining LightGBM with XGBoost (which drops the geohash column and instead leans on lat/lon and geo_mean) captured complementary errors. XGBoost carried the larger weight (0.594) in model C.

- * Prediction blending. Averaging independent models (B and C, 50/50) gave the final lift to 91.25092 -- larger than any single-model change.
- * The two time-of-day profile features (road_tod_mean, lanes_tod_mean) in B.

What did NOT help (tried and dropped):

- * Extra engineered features beyond road_tod / lanes_tod (e.g. morning / shift features in v3) -- they hurt or were flat on the daytime proxy.
- * Hyperparameter tuning (explored with Optuna) -- no reliable gain over the hand-set v1 parameters.
- * Target reformulation (loglp / sqrt transforms) -- the data is clean and the skew is genuine high-demand signal, so the RAW target was best for RMSE.
- * Class / seed tricks and heavier seed bagging (v4 / v5) -- essentially flat.

Conclusion: the data was clean, the target was best left raw, and the gains came from a trustworthy validation proxy plus diversity-driven blending rather than from feature inflation or aggressive tuning.

6. TOOLS

- * Language : Python 3.14 (developed and validated on 3.14.3, Windows).
- * Data / math : pandas, numpy.
- * Modeling : LightGBM and XGBoost (the final two-model components); scikit-learn for utilities and metrics. CatBoost was explored but its selected blend weight was exactly 0.0, so it is omitted (dropping a zero-weight model changes nothing and trims the dependency list).
- * Tuning : Optuna was explored for hyperparameter search (did not beat the v1 settings, so the baked-in parameters are used).
- * Diagrams : matplotlib (see docs/diagrams/).
- * Determinism : fixed, hard-coded seeds; seed-averaging over fixed seed lists; no runtime search.

Exact pinned versions are listed in `requirements.txt` (numpy, pandas, scikit-learn, lightgbm, xgboost).

7. REPRODUCIBILITY

The submission is deterministically regenerated by a single self-contained script.

How to run (from inside this folder):

```
python reproduce.py
```

- * It auto-locates the data at ../dataset/train.csv and ../dataset/test.csv (and probes a few other common locations; on Google Colab it prompts for an upload). See COLAB_GUIDE.md and Gridlock_Reproduce_Colab.ipynb for the notebook walkthrough.
- * Output: `submission\_BC.csv` (41778 rows; columns [Index, demand]; demand

clipped to [0, 1]) written into this folder.

- * No runtime search: all hyperparameters, boosting round counts, seed lists, and blend weights are baked in as constants. B is averaged over seeds [42..47]; C over seeds [42, 143, 244, 345].

Verified deterministic: regenerating from `reproduce.py` reproduces the committed file to floating-point tolerance -- maximum absolute difference $\sim 4.44e-16$ versus `submission\_BC\_reference.csv`, Pearson correlation 1.0 across all 41778 rows (full details in VERIFICATION.md).

8. FILE MANIFEST (this package)

APPROACH.txt	This document (approach, features, tools).
README.md	Package overview and run instructions.
reproduce.py	Deterministic regenerator (run this).
requirements.txt	Pinned dependency versions.
eda.py	Standalone EDA pipeline.
eda_report.txt	EDA output (generated by eda.py).
COLAB_GUIDE.md	Google Colab walkthrough.
Gridlock_Reproduce_Colab.ipynb	Colab notebook for reproduction.
VERIFICATION.md	Independent reproducibility verification.
submission_BC.csv	The winning prediction file (score 91.25092).
submission_BC_reference.csv	Validated reference for the match check.
reproduce_run.log	A captured reproduce.py run log.
docs/Gridlock_Solution_Report.pdf / .docx	Full written report.
docs/build_report.py	Script that builds the report and diagrams.
docs/diagrams/*.png	EDA, pipeline, timeline, validation, and model evolution figures.
